3강. Array 객체 내장 함수 => 가장 중요한 객체 : 서버로부터 받아온 데이터 처리 용도로 많이 사용

① toString( ) : 쉼표를 기준으로 하나의 문자열로 변환

```
let brands = ["애플", "구글", "아마존", "마이크로소프트", "메타"];
console.log(brands.toString());  //애플,구글,아마존,마이크로소프트,메타
```

② join( ) : 파라미터로 전달된 문자를 이용해서 하나의 문자열로 변환하며 실무에서 많이 사용

```
console.log(brands.join("*")); //애플*구글*아마존*마이크로소프트*메타
console.log(brands.join("")); //애플구글아마존마이크로소프트메타
```

③ pop() : 배열의 마지막 요소 반환하고 배열에서 제거

```
console.log(brands.pop()); //메타
console.log(brands); // ['애플', '구글', '아마존', '마이크로소프트']
```

④ push() : 배열의 마지막 요소 추가, join과 많이 사용

```
brands.push("테슬라");
console.log(brands); //['애플', '구글', '아마존', '마이크로소프트', '테슬라']
```

⑤ shift() : 배열의 첫 요소 반환하고 배열에서 제거 => 실무에서 많이 사용

//서버 프로그램에서 많이 사용 => 요청을 배열에 쌓아두었다가
//이벤트 큐에 순서대로 정리된 목록에서 1번 요청, 2번 요청...하나씩 꺼내서 작업을 처리

⑥ unshift() : 배열의 맨 앞에 요소를 추가하고 배열의 길이를 반환

실무활용 => 음료를 데이터베이스에서 가져와서 일정한 기준으로 분류하여 활용한다고 가정해보자. 이 때 등록하는 화면과 조회하는 화면이 다를 수 있다. 예를 들어 등록하는 화면에는 전체 option 이



필요하지 않지만, 조회하는 화면은 전체 option



이 필요하다.

그러나 같은 팀의 동료가 제공하는 함수 또는 디자인은 맨 위 칸이 없는 상태로 다음과 같이 제공될 가능성이 높다.

```
const loadDrinkType = ( ) => {
        const types = [
          { text: "이온음료", code: "A" },
          { text: "커피", code: "B" },
          { text: "탄산음료", code: "C" },
        ];

        const h = []; //option 이 들어가는 배열

        for (const type of types) {
          h.push("<option value=" + type.code + ">" + type.text + "</option>");
        }
        return h;
    } //end of function

  const selDrinkType = loadDrinkType(); //화면 열리자마자 실행
  document.getElementById("selDrinkType").innerHTML = selDrinkType.join("");
```

조회 등의 기능에서처런 맨 위 칸에 '전체'를 위한 칸이 필요하다면 다음 푸른 부분처럼 소스 코드를 수정하지 말고 unshift( ) 함수를 이용하면 주어진 함수 코드를 수정하지 않고 새로운 option 을 추가할 수 있어 매우 효율적이다.

```
function loadDrinkType(needAll) {
        const types = [
          { text: "이온음료", code: "A" },
          { text: "커피", code: "B" },
          { text: "탄산음료", code: "C" },
        ];

        const h = []; //option 이 들어가는 배열

        for (const type of types) {
          h.push("<option value=" + type.code + ">" + type.text + "</option>");
        }

        if (needAll) {
          h.unshift('<option value=""></option>');//전체 선택
        }
        document.getElementById("selDrinkType").innerHTML = h.join("");
      } //end of function
      loadDrinkType(true);
```

```
function loadDrinkType() {
        const types = [
          { text: "이온음료", code: "A" },
          { text: "커피", code: "B" },
          { text: "탄산음료", code: "C" },
        ];

        const h = []; //option 이 들어가는 배열

        for (const type of types) {
          h.push("<option value=" + type.code + ">" + type.text + "</option>");
        }
        return h;
} //end of function
const selDrinkType = loadDrinkType(); //화면 열리자마자 실행
selDrinkType.unshift('<option value=""></option>'); //unshift()함수를 이용하여 앞칸을 빈칸을 삽입
document.getElementById("selDrinkType").innerHTML = selDrinkType.join("");
```

```
//간단히 해결할 수는 있지만, 실제로 비용이 들 수 있다. 배열 요소의 갯수가 많다면 배열 요소를 전부 하나씩 뒤로 밀어야 하기 때문이다.
const loadDrinkType2 = (plus) => {
      const types = [{text:"이온음료", code :"A"}, {text:"커피", code :"B"}, {text:"탄산음료", code :"C"}]

      const tags = [];
      for(type of types){
                tags.push("<option value =  " + type.code + ">" + type.text + "</option>")
      }

     if(plus){
          tags.unshift("<option value = " + plus.code + ">"+ plus.text + "</option>");
     }
          return tags;
 }
 const selDrinkType2 = loadDrinkType2({text:"전체", code :"ALL"}); //태그가 들어있는 배열 변수
 //const selDrinkType2 = loadDrinkType2({text:"쥬스", code :"D"}); //태그가 들어있는 배열 변수
 document.getElementById("selDrinkType2").innerHTML = selDrinkType2.join(""); //배열의 요소를 문자열로 합쳐서 화면에 뿌리기
```

⑦ splice(1, 2, 3) : 배열의 특정 위치에 새로운 요소를 추가

```
//기존 요소를 삭제하면서 추가할 수 있다.
//1. 새로운 요소를 추가할 인덱스 2. 요소를 추가하기 전에 삭제할 요소 수 3. 추가할 요소

brands.splice(1, 0, "개발자의 품격", "더 그레잇"); //추가
console.log(brands); //['삼성전자', '개발자의 품격', '더 그레잇', '구글', '아마존', '마이크로소프트', '테슬라']
```

```
brands.splice(1, 2); //제거
console.log(brands); // ['삼성전자', '구글', '아마존', '마이크로소프트', '테슬라']

//직관이 아닌 로직으로 '구글'을 찾아서 '구글'뒤에 '쿠팡'을 splice해 보자. => 배열도 문자열처럼 indexOf, lastIndesOf가 적용된다.
brands.splice(brands.indexOf('구글')+1, 0, "쿠팡");
console.log(brands)
```

⑧ **concat() : 두 개 이상의 배열을 하나의 배열로 결합**

```
let arr1 = ["A", "B"];
let arr2 = ["C", "D"];
let arr3 = ["E", "F", "G"];
let arr4 = arr1.concat(arr2, arr3);
console.log(arr4); //['A', 'B', 'C', 'D', 'E', 'F', 'G']
let arr5 = [...arr1, ...arr2, ...arr3]; //배열 spread operator(배열 전개 연산자)
console.log(arr5);
```

⑨ **slice( ) : 배열의 요소를 잘라내서 배열 형태로 반환**(시작 인덱스, 마지막인덱스 -1)

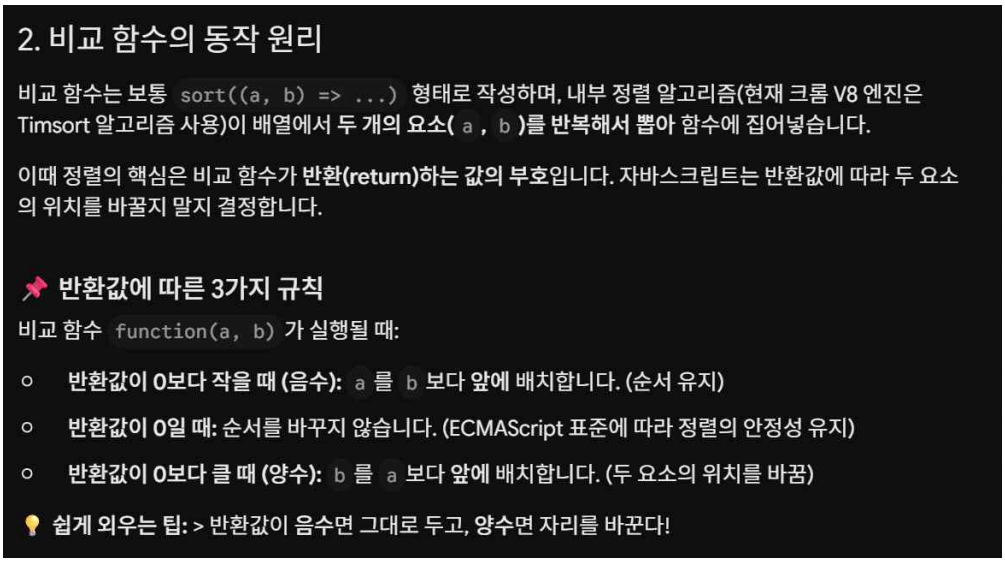
```
console.log(arr4.slice(1, 2)); //[B]
```

⑩ **sort() : 배열의 요소를 정렬해주는 함수 => 문자열 기준**

```
console.log(brands.sort()); //오름차순으로 정렬

let points = [40, 100, 1, 5, 25, 10];
console.log(points.sort()); //[1, 10, 100, 25, 40, 5] => 문자로 인식
```

//숫자의 오름차순 : 별도의 알고리즘을 동작시킨다. 함수를 매개변수로 갖는 엔진이 갖고 있는 내장함수이다.



```
const ascPoints = points.sort(function (a, b) {
  //a-40, b-100  [40, 100, 1, 5, 25, 10]
  //a-100, b-1   [40, 1, 100, 5, 25, 10]
  //a-100, b-5   [40, 1, 5, 100, 25, 10]
  //a-100, b-25  [40, 1, 5, 25, 100, 10]
  //a-100, b-10  [40, 1, 5, 25, 10, 100]
  //a-40, b-1    [1, 40, 5, 25, 10, 100]
  //a-40, b-5    [1, 5, 40, 25, 10, 100]
  //자리 바꿈이 더 이상 발생하지 않을 때까지 반복 => sort() 함수가 하는 일

  // if (a > b) return 1; //양수이면 자리를 바꾼다.
  // else if (a < b) return -1; //음수이면 자리를 바꾸지 않는다.
  // else 0;

  return a - b; //위의 세 문장을 한 문장으로 나타내면
});
console.log(ascPoints);
```

```
//숫자의 내림차순
const descPoints = points.sort(function (a, b) {
   return b - a;
});
console.log(descPoints); // [100, 40, 25, 10, 5, 1]
```

⑪ filter( ) : 배열에서 특정 조건에 맞는 요소만 배열로 리턴 => 원배열은 변하지 않는다.

```
const drinkList = [
   { name: "오렌지", price: 1000 },
   { name: "파워에이드", price: 1200 },
   { name: "커피", price: 700 },
   { name: "보리음료", price: 1200 },
   { name: "코카콜라", price: 1800 },
];

let availableProduct = [ ];
const inputCoin = 1000;

for (const drink of drinkList) {
  if (drink.price <= inputCoin) {
```

```
      availableProduct.push(drink);
    }
  }
  console.log(availableProduct);
  availableProduct = drinkList.filter((drink) => {
    return drink.price <= inputCoin;
  });
  console.log(availableProduct);
```

⑫ map() : 배열의 모든 요소를 하나씩 돌면서 실행하고 새로운 배열을 생성하는 함수
//기존 배열은 변화하지 않는다.

```
// 다음 예제에서 lastName + firstName 을 fullName으로 리턴한다고 가정해보자.
let userList = [
  {
    firstName: "재석",
    lastName: "유",
    email: "you@gmail.com",
  },
  {
    firstName: "종국",
    lastName: "김",
    email: "kim@gmail.com",
  },
  {
    firstName: "지효",
    lastName: "송",
    email: "song@gmail.com",
  },
];

let newUserList = userList.map((user) => {
  return {
    fullName: user.lastName + user.firstName,
    email: user.email,
  };
});
console.log(newUserList);
```

⑬ reduce(p1, p2, p3, p4) : 배열요소의 누적 합을 구하는 함수, 또는 특정한 조건에 해당하는 값만 추출하는 함수

```
//p1 : accumulator => 누적 값
//p2 : currentValue => 현재 값
//p3 : currentIndex => 현재 배열 인덱스, 생략 가능
//p4 : arr => 전체 배열, 생략 가능

/* 형식
  배열.reduce((누적값, 현재값, 인덱스, 원본배열) => {
      return 다음 누산기에 넘겨줄 값
  }, 초기값)
*/

points = [40, 100, 1, 5, 25, 10];

let sum = 0;
for (const point of points) {
  sum += point;
}
console.log(sum);

//점수의 합을 구해보자.
sum = points.reduce((sum, point) => {
  return (sum += point);
}, 0); //sum의 초기값은 0
console.log(sum); //101

//음료수 가격의 합을 구해보자.
drinkList = [
  { name: "오렌지", price: 1000 },
  { name: "파워에이드", price: 1200 },
  { name: "커피", price: 700 },
  { name: "보리음료", price: 1200 },
  { name: "코카콜라", price: 1800 },
];

let drinkTotal = drinkList.reduce((total, drink) => {
  return (total += drink.price);
}, 0); //drinkTotal의 초기값은 0
console.log(drinkTotal); //5900
```

```
//성이 '김'인 사람을 골라내보자.
//reduce()을 이용하여 성이 김씨인 사람을 찾아보자. 굳이???? filter로 하면 훨씬 편함!!!!!

 userList = [
   {
     firstName: "재석",
     lastName: "유",
     email: "you@gmail.com",
   },
   {
     firstName: "종국",
     lastName: "김",
     email: "kim@gmail.com",
   },
   {
     firstName: "지효",
     lastName: "송",
     email: "song@gmail.com",
   },
 ];

 //p1 : accumulator => 누적값
 //p2 : currentValue => 현재 값
 //p3 : currentIndex => 현재 배열 인덱스, 생략 가능
 //p4 : arr => 전체 배열, 생략 가능

 let kims = userList.reduce((users, user) => {
   if(user.lastName === "김"){
     users.push(user);
   }
   return users;
 }, []);

 console.log(kims);

 let kims2 = userList.filter((user) => {
   return user.lastName === "김";
 });

 console.log(kims2);
```

4강. Date 객체 내장 함수

현재의 년, 월, 일, 시, 분, 초, 밀리초 데이터를 얻어오는 객체

① 현재 날짜 객체 얻어오기

```
//날짜 - 년, 월, 일, 시, 분, 초, 밀리초
let now = new Date();
console.log(now);
```

② 사용자가 날짜를 지정해서 객체를 얻어오기

```
//날짜를 사용자가 지정한 경우 : 년, 월(0~), 일, 시간, 분, 초, 밀리초로 만들기
let d = new Date(2025, 4, 29, 19, 34, 20, 0);
console.log(d);
```

③ get() 함수의 활용

```
//get 함수의 활용
const year = now.getFullYear(); //년도
let month = now.getMonth(); //월(0~)
const day = now.getDate(); //일
const dayName = now.getDay(); //요일(일요일 0)
const hour = now.getHours(); //시간
const minute = now.getMinutes(); //분
const second = now.getSeconds(); //초
const millisecind = now.getMilliseconds(); //밀리초
```

④ set() 함수의 활용 => 거의 사용하지 않는다.

```
//set 함수의 활용 => get 함수에 대응하여 다 있다. 그러나 거의 사용하지 않는다.
now.setFullYear(2021);
console.log(now);
```

⑤ Date 객체의 활용 1. => 특정한 포맷으로 출력하고자 할 경우

```
const today= year + "-" + month.toString().padStart(2, 0) + "-" + day.toString().padStart(2, 0);
console.log(today);

//오늘 날짜 body(endDate)에 뿌리기
document.getElementById("endDate").value = today;
```

※ 참고 : 7일전을 가장 쉬운 방법으로 구해보자.(= day-7)

```
//월까지 생각해야 하므로 거의 불가능하다.
//에러 발생할 가능성이 높다. .. 만일 오늘 날짜가 5일이라면 5-7???  => 에러가 발생한다.
let before7days = new Date(year, month, day - 7);
console.log('7일전 : ', before7days);
```

⑥ 날짜별 조회 기능 구현(기준이 되는 시간 구하기)

```
//1970-1-1
let d2 = new Date(0); //밀리초를 입력해서 날짜를 구하는 것도 가능
console.log(d2); //Thu Jan 01 1970 09:00:00 GMT+0900 (한국 표준시)

//하루를 밀리초로 연산 : 24*60*60*1000
console.log(new Date(24 * 60 * 60 * 1000)); //1970-1-2

//오늘을 밀리초로 구하기 : getTime()함수는 특정 날짜와 시간을 밀리초로 계산
currentMilliseconds = now.getTime()
console.log(currentMilliseconds);
```

⑦ 오늘을 기준으로 며칠 전, 후를 구하는 함수 만들기

```
const getIntervalDate = ((day) =>{
    let now = new Date();
    let dayMilliseconds = 24 * 60 * 60 * 1000; //하루
    let intervalDate = now.getTime() + day * dayMilliseconds;
    return new Date(intervalDate);
});

console.log(`3일전 : ${getIntervalDate(-3)}`); //3일전
console.log(`7일전 : ${getIntervalDate(-7)}`);//7일전
console.log(`30일전 : ${getIntervalDate(-30)}`);//30일전
console.log(`7일후 : ${getIntervalDate(7)}`);//7일 후
```

⑧ Date 객체의 활용 2. => 날짜별 조회 기능 구현(실습예제 참조)

5강. Set 객체 내장 함수

- **ECMA 6버전에서 추가된 객체**
- 배열과 매우 유사하나, 유일한 요소만 갖는다.
- 따라서 기존의 긴 소스들을 간단하게 줄일 수 있다.

- 다음은 좋아하는 음식을 단답형으로 입력하는 설문 조사로 사용자가 입력한 음식명을 각각 배열과 Set로 저장한다고 가정해보자.
- let foods = ["햄버거", "김치찌개", "된장찌개", "제육볶음", "김치찌개"] ; //사용자가 입력한 **음식 종류**를 저장한 배열

	배열	Set

유일한 음식명만 뽑기 방법1.
 	
객체의 키는 유일하다는 개념을 사용
	let foods = ["햄버거", "김치찌개", "된장찌개", "제육볶음", "김치찌개"];
	
	let oFood = { };//키와 값을 한쌍으로 가짐
	for (const food of foods) {
	oFood[food] = "";
	}
	console.log(oFood); **//{햄버거: '', 김치찌개: '', 된장찌개: '', 제육볶음: ''}**
	
	let uniqueFoods = [ ];
	for (const key in oFood) {
	//for in은 유일한 것만 가져온다
	uniqueFoods.push(key);
	}
	console.log(uniqueFoods); **//['햄버거', '김치찌개', '된장찌개', '제육볶음']**
		let foodSet = new Set();
		foodSet.add("김치찌개");
		foodSet.add("된장찌개");
		foodSet.add("김치찌개");
		foodSet.add("제육볶음");
		foodSet.add("햄버거");
		foodSet.add("된장찌개");
		
		foodSet.forEach(function (food) {
		console.log(food);
		});
		
		=> 이미 set은 생성 단계에서 유일한 값만 가진다.

- 유일한 음식명만 뽑기 방법2. filter함수를 이용하는 방법	let foods = ["햄버거", "김치찌개", "된장찌개", "제육볶음", "김치찌개"];
	
	//현재 비교중인 인덱스
	let uniqueFood = foods.filter((food, index)=>{
	// if (index !== foods.indexOf(food)){
	// return food;
	// }
	if (index === foods.indexOf(food)){
	return food;
	}
	});
	console.log(uniqueFood);
		let uniqueFood2 = new Set(foods);
		console.log(uniqueFood);
		
		console.log(uniqueFood2.has("햄버거"));
		
		=> 이미 set은 생성 단계에서 유일한 값만 가진다.

	배열	Set

특정 데이터를 찾고자 하는 경우	for (const food of uniqueFoods) {
	  if (food === "김치찌개") {
	  console.log("김치찌개가 있습니다. ");
	  }
	}
		console.log(**foodSet.has**("김치찌개"));

특정 데이터를 삭제하고 싶은 경우	let indexKim;
	for (food of uniqueFoods) {
	  indexKim = uniqueFoods.indexOf("김치찌개");
	 }
	uniqueFoods.**splice**(indexKim, 1);
	console.log(uniqueFoods);
		foodSet.**delete**("김치찌개");
		console.log(foodSet);
		foodSet.**clear()**; //전체 내용을 삭제

- 필요한 데이터의 특성(고유한 데이터)에 따라 Set 을 사용하면 많은 수고를 줄일 수 있다.
- 파이썬처럼 add, has, delete, clear 등의 함수를 내장하고 있다.

6강. Map 객체 내장 함수

- key와 value로 이루어진 객체, 마치 Object 와 유사
- key에 문자가 아닌 값이 올 수 있으므로, 키가 문자열이 아닌 object를 만들고자 할 경우 대체제로 활용이 가능
- 파이선의 딕셔너리와 유사
- Object 와 Map의 차이

	Object	Map
key	문자만 가능	모든 타입 가능
size property	속성 제공 X	size 속성 제공
for ...in...의 순서	보장 X	보장 O

-

```
<script>
    //object
    let oPerson = {};
    oPerson.name = "홍길동";
    oPerson.email = "hong@gmail.com";
    oPerson.phone = "010-0000-0000";
    oPerson["180"] = 120;  //키, 몸무게

    //객체는 사이즈를 확인하기 위해 count 변수를 사용해야 한다.
    //객체는 출력 순서가 보장되지 않는다.
    let count = 0;
    for (const key in oPerson) {
      console.log(`${key} : ${oPerson[key]}`);
      count++;
    }
    console.log(`oPerson의 데이터의 갯수 : ${count}`);

    //Map의 생성
    //1. set 명령어
    let personMap = new Map();
    personMap.set("name", "홍길동");
    personMap.set("email", "hong@gmail.com");
    personMap.set("phone", "010-0000-0000");
    personMap.set(180, 120);

    //forEach를 사용하면 매우 편리하다.
    //map은 출력 순서가 보장된다.
    personMap.forEach((value, key) => {
      console.log(`${key} : ${value}`);
    });
```

```
 //2.key만 골라내기
 for (let key of personMap.keys()){
  console.log(key);
 }

 //3.value만 골라내기
  for (let value of personMap.values()){
  console.log(value);
 }
  //4. map은 size 속성을 지원한다.
  console.log(personMap.size); //3,property

  //5. get() : 값을 읽어오기
  console.log(personMap.get("name"));

  //6. has() : 키의 존재여부 알아내기
  console.log(personMap.has("email")); //키의 존재여부
  console.log(personMap.has("홍길동")); //false

  //7. delete() : 해당 item 삭제
  personMap.delete("email");
  console.log(personMap.size); //2

  //8. clear() : 맵 전체 삭제
  //personMap.clear();
</script>
```

7강. Math 객체 내장 함수

① round() - 반올림처리 => 실무에서 사용 많음

```
console.log(Math.round(4.9)); //5
console.log(Math.round(4.4)); //4
```

② ceil() - 올림처리 => 실무에서 사용 많음 => 페이징처리

```
console.log(Math.ceil(4.7)); //5
console.log(Math.ceil(4.2)); //5

// 실무에서 활용 : 페이징처리
//페이징처리시 사용
//게시판에 작성한 글이 51개 등록되었다고 가정
//게시판 페이지는 한 페이지당 10개의 글 목록을 보여줌
//총 페이지는 Math.ceil(전체글 수/페이지당 글 수)
let total = 51;
let countPerPage = 10;
let totalPage = Math.ceil(total / countPerPage); //6페이지
console.log(totalPage); //6
```

③ floor() : 내림

```
console.log(Math.floor(4.7)); //4
```

④ trunc( ) : 소수점 이하는 무조건 버림

```
console.log(Math.trunc(4.7)); //4

// floor() vs trunc() : 양의 소수점은 문제가 없으나 음의 소수점은 문제가 발생
console.log(Math.floor(-4.2)); //-5
console.log(Math.trunc(-4.2)); //-4
```

⑤ sign() : 양수이면 1, 음수이면 -1, 0 => 거의 사용 안 함

```
console.log(Math.sign(-4)); //-1
console.log(Math.sign(2345));
```

⑥ pow() : 제곱값

```
console.log(Math.pow(8, 2)); //64
```

⑦ sqrt() : 제곱근값

```
console.log(Math.sqrt(64)); //8
```

⑧ abs() : 절대값

```
console.log(Math.abs(-4.7)); //4.7
```

⑨ min() : 최소값

```
console.log(Math.min(4, 2, 5, 6, -8, -22)); //-22
```

⑩ max(): 최대값

```
console.log(Math.max(2, 4, 5, 6, 7));
```

⑪ random() : 랜덤값 => 0보다 크거나 같고 1보다 작은 값을 무작위로 발생 => 가장 실무에서 많이 사용

```
console.log(Math.random());

//활용1. 0~9
  console.log(Math.random() * 10); //0보다는 크고 10보다 작은 수
  console.log(Math.floor(Math.random() * 10)); //내림
//활용2. 1~10
  console.log(Math.floor(Math.random() * 10) + 1); //0~9를 구해서 +1
//활용3. 1~100
 console.log(Math.floor(Math.random() * 100) + 1);

//활용4. 시작값과 종료값을 주면 사이값을 random하게 나오게 하는 함수를 작성하여 활용해보자
//위의 3가지 사례를 min, max 에 넣어서 이해해보자.
      function getRandomInteger(min, max) {
        return Math.floor(Math.random() * (max - min + 1)) + min;
      }
      console.log(getRandomInteger(1, 10));
```